

- importance of lines, 161
- legends, 137–139, 151
- modular decomposition diagrams, 272
- notation, 136, 138–139, 151, 265, 278, 280
- patterns in, 138–139
- sequence diagrams, 278, 280
- whiteboard jams, 255–257
- divergence and design exploration, 63–65, 118
- divide and conquer activity, 239–243
- do loop, *see* think, do, check loop
- do-make-check cycle, 285
- documentation, *see also* architecture descriptions
 - architecture decision records, 146, 155, 260–262, 274
 - architecture haiku, 264
 - ASR Workbook, 60–62, 148
 - building model into code, 107
 - code comments, 111
 - design plan, 37
 - event specification, 89
 - functional requirements, 58
 - greatest hits reading list, 267
 - moth ball document, 186
 - paths not taken, 155, 228, 231, 274
 - Project Lionheart, 186
 - sanity check, 304
- A documentation framework for architecture decisions*, 262
- domain
 - activities, 225
 - architecture flipbook, 229
 - Component Responsibility Cards (CRC cards), 232–235
 - concept map, 236–238
 - event storming, 244–248
 - group poster activity, 250
 - models and, 105, 108
 - names and, 105
 - selecting architecture and, 65
- Domain-Driven Design: Tackling Complexity in the Heart of Software*, 238
- dot voting
 - design studios, 120
 - empathy maps, 197
 - group poster activity, 250
 - mini-Quality Attribute Workshop (mini-QAW), 211
 - quality attributes web, 208
- drawings and sketches, *see also* visualizing design
 - architecture evaluation, 160, 168
 - architecture evaluation workshops, 168
 - architecture flipbook, 228–231
 - Component Responsibility Cards (CRC cards), 234
 - group poster activity, 249–251
 - personify the architecture, 226
 - practice, 120
 - question-comment-concern activity, 299
 - risk storming, 301
 - round-robin design activity, 252–254
 - sketch and compare activity, 168, 311–313
 - tips for, 136–140
 - whiteboard jams, 255–257
- E**
- education, *see also* team development
 - architecture briefings, 286
 - architecture design studios, 124
 - architecture flipbook, 228–231
 - event storming, 244
 - information sessions, 181
 - just-in-time education, 124
 - question-comment-concern activity, 298
 - scaffolding, 180
- elaboration phase, 37
- element-responsibility catalog, 74, 130
- element-responsibility views, 130
- elements
 - in architecture descriptions, 149
 - assigning functional responsibilities, 73–75
 - center of competence pattern, 94
 - Component Responsibility Cards (CRC cards), 232–235
 - creating structures, 8
 - defined, 8
 - element-responsibility catalog, 74, 130
 - element-responsibility views, 130
 - enforcing relations, 109
 - exercises, 9
 - layers pattern, 81
 - models and, 105, 109
 - multi-tier pattern, 92
 - naming, 105
 - open source contribution pattern, 95
 - organizing code to make patterns obvious, 109
 - pattern visualizations, 139
 - pipe-and-filter pattern, 84
 - ports and adapters pattern, 82
 - publish-subscribe pattern, 88
 - selecting architecture and, 64, 73–75
 - service-oriented pattern, 86
 - shared-data pattern, 90
 - as term, 9
- empathy
 - active listening, 61
 - map, 150, 195–198
 - with stakeholders, 16, 39–46, 191, 195–198
- empathy map, 150, 195–198
- enforcing relations, 109
- environment context in quality attribute scenarios, 53
- erosion in architectural issues rainbow, 172
- erosion, architectural, *see* architectural drift

- evaluate design mindset
 - activities and exercises, 285–313
 - defined, 20
 - do-make-check cycle, 285
 - exercise, 20
 - icons, xvi
 - using, 18–24
 - using risk to choose, 34
 - evaluating architecture, *see* architecture evaluation
 - evaluation pyramid, 171–172
 - event bus, 88
 - event sourcing, 248
 - event specification, 89
 - event storming, 244–248
 - exceptions and code comments, 111
 - exercises, *see also* activities
 - about, xvii
 - active listening, 61
 - architecture design studios, 120
 - architecture evaluation, 166
 - business goals, 46
 - delegating authority, 184
 - diagrams, 140
 - elements, relations, and structures, 9
 - in explore design mindset, 20
 - meta-model, 106
 - models, 106
 - paths not taken, 156
 - quality attribute scenarios, 55
 - stakeholders, 43, 46
 - team development, 184
 - experiments, treating solutions as, 28
 - experts
 - architecture evaluation workshops, 167
 - center of competence pattern, 93
 - event storming, 244–248
 - open source contribution pattern, 95
 - exploration plan, 241
 - explore design mindset
 - activities and exercises, 225–257
 - breadth-first vs. depth-first approach, 241
 - defined, 19
 - divergence and convergence, 118
 - exercise, 20
 - icons, xvi
 - patterns and quality attributes, 69–71
 - using, 18–24
 - using risk to choose, 34
 - Extreme Programming Explained: Embrace Change*, 281
- ## F
- Fairbanks, George, xiii, xvii, 35, 107, 150, 231, 263
 - features
 - mini-Quality Attribute Workshop (mini-QAW), 212–213
 - software architect's role, 4
 - feedback, *see also* architecture evaluation; code reviews; peer reviews; think, do, check loop
 - architecture briefings, 286–288
 - architecture design studios, 115, 117
 - divide and conquer activity, 239
 - team development, 181
 - fighting the framework, 88
 - filter pattern, *see* pipe-and-filter pattern
 - flipbook, architecture, 106, 228–231, 238
 - focal point for remote architecture design studios, 125
 - follow-up
 - architecture design studios, 115, 118
 - architecture evaluation workshops, 166, 170
 - code reviews, 290
 - mini-Quality Attribute Workshop (mini-QAW), 214
 - sketch and compare activity, 312
 - Foote, Brian, 96
 - fork and pull workflow and enforcing element relations, 110
 - formal architecture description method, 145–148
 - formal models, 146
 - Fowler, Martin, 13
 - frameworks
 - architectural mismatch, 88
 - domain model, 108
 - fighting, 88
 - selecting architecture and, 65
 - functional requirements, *see* architecturally significant requirements (ASRs); influential functional requirements
- ## G
- gRPC in service-oriented pattern, 86
 - Game of Life, Conway's, 106
 - Gang of Four, 80
 - gaps
 - in architectural issues
 - rainbow, 172
 - concept gaps, 237
 - model-code gap, 107
 - question-comment-concern activity, 298
 - sanity check, 304
 - glossary, term, 148
 - The Goal Question Metric (GQM) Approach*, 199
 - Goal-Question-Metric (GQM) Workshop, 59, 199–201, 296
 - goals, *see also* business goals
 - architecture design studios, 115–117, 123
 - Goal-Question-Metric (GQM) Workshop, 59, 199–201, 296
 - Good Naming Is a Process, Not a Single Step*, 105
 - Goodwin, Kim, 202
 - GQM (Goal-Question-Metric) Workshop, 59, 199–201, 296
 - GQM tree, 200
 - Graylog, 297
 - greatest hits reading list, 267
 - group dynamics/management
 - architecture design studios, 118, 121–126
 - divide and conquer activity, 241

group poster activity, 119, 249–252
 groupthink, 121
 guide rails, architectural, 180

H

haiku, architecture, 146, 184, 263, 274
 Halloway, Stuart, 286, 288
 HART principles, 15–18
 Hexagonal Architecture, 82
 Hilliard, Rich, 262
 Hohpe, Gregor, 101
How to Win Friends and Influence People, 61
 human rule, 16
 human-centered design
 about, xv
 architecture descriptions, 152
 architecture design studios, 120
 benefits, 13, 15
 business goals, 44
 collaboration, 13, 16
 human rule, 16
 personify the architecture, 120, 184, 226
 software architect's role, 185
 software architecture design as, 4
 tangibility rule and, 18
 Hystrix Dashboard, 297

I

inception deck, 59, 184, 269–271
 influential functional requirements, *see also* architecturally significant requirements (ASRs)
 activities, 59, 191, 193
 architecture flipbook, 230
 architecture haiku, 263
 assigning to elements, 73–75
 calculator app example, 57
 classes of, 56
 Component Responsibility Cards (CRC cards), 232–235
 concept map, 237
 defined, 49, 56

mini-Quality Attribute Workshop (mini-QAW), 212
 vs. quality attributes, 52
 selecting architecture and, 73–75
 information sessions and team development, 181
 inner layers, 82
 insights in architecture evaluations, 160, 164, 169
 instructional scaffolding, 180
 instrumentation and observing behavior, 295–297
 intent
 architecture descriptions, 155
 models, 100
 names and, 105
 interface segregation, 76
 interviewing stakeholders, 59, 202–204, 220
Introducing Event Storming: An Act of Deliberate Collective Learning, 244
 inversion of control, 291
 ISO/IEC/IEEE 42010 standard
 architecture decision records, 262
 architecture descriptions, 153
 SAD templates, 147
 iterating
 architecture design studios, 115, 117–119
 length, 21
 speed, 28
 think, do, check loop, 21–24, 35

ivory tower software architects, 16, 40

J

js-sequence-diagrams, 279
Just Enough Software Architecture: A Risk-Driven Approach, 35, 107
 just-in-time education, 124
 JVM, 297

K

Krutchen, Phillippe, 154
 Kua, Patrick, xvii, 183, 286
 Kubernetes, 132

L

A Laboratory for Teaching Object-Oriented Thinking, 232
 last responsible moment, 75
 layers
 inner layers, 82
 layers pattern, 80–81, 104
 ports and adapters pattern, 82
 layers pattern, 80–81, 104
 leadership, *see* team development
Lean Software Development: An Agile Toolkit for Software Development Managers, 75
 learning, *see* education; team development
 legends
 diagrams, 137–139, 151
 event storming, 247
 group poster activity, 250
 Leifer, Larry, 15
Less is more with minimalist architecture, 17
 life line in sequence diagrams, 278
 lines, *see* box-and-line diagrams
 Liskov substitution, 76
 list assumptions activity, 59, 205–206
 listening
 active, 61
 interviewing stakeholders, 202–204
 response measure straw man, 220
 team development, 183
 logging platforms, 297
 LogStash, 297

M

mad lib, Point-of-View (POV), 44, 215–218
 make design mindset
 activities to make design tangible, 259–283
 architecture design studios, 114
 benefits, 259
 defined, 19
 exercise, 20
 icons, xvi

- using, 18–24
 - using risk to choose, 34
 - Making Metaphors That Matter*, 281
 - Malan, Ruth, 17
 - Management 3.0, 184
 - Management 3.0: Leading Agile Developers, Developing Agile Leaders*, 181
 - Managing for Happiness: Games, Tools, and Practices to Motivate Any Team*, 184
 - mapping structures, *see* allocation structures
 - mapping views, 133
 - Marathon, 132
 - Mars Rover, 53
 - Master of the SAD, 148
 - matching sanity check, 306
 - may contribute to relation, 95
 - Meinel, Christoph, 15
 - message system
 - in sequence diagrams, 279
 - in service-oriented pattern, 86
 - meta-model, 101–107, 111, 136
 - metaphors, system, 146, 184, 281–283
 - metrics
 - Goal-Question-Metric (GQM) Workshop, 59, 199–201, 296
 - observing behavior activity, 295
 - microservice architectures
 - enforcing relations and, 110
 - observing behavior activity, 296
 - sequence diagrams, 279
 - milestones, 37
 - mindsets, *see* design mindsets
 - mini-Quality Attribute Workshop (mini-QAW), 59, 210–214, 220
 - minimalism
 - minimalist architecture, 17
 - refinement views, 132
 - minimalist architecture, 17
 - mismatch, architectural, 88
 - model-code gap, 107
 - models, 99–112, *see also* visualizing design
 - advantages, 100
 - architecture flipbook, 228–231
 - building into code, 107–111
 - curiosity cycle, 102
 - formal models, 146
 - generating from code, 111
 - make design mindset, 19
 - meta-model, 101–107, 111, 136
 - Project Lionheart, 111
 - software architect's role, 185
 - using, 99–112
 - modular decomposition diagrams, 272
 - module structures
 - big ball of mud pattern, 96
 - vs. components, 9
 - defined, 8
 - enforcing element relations, 109
 - exercise, 9
 - layers pattern, 81
 - organizing code to make patterns obvious, 109
 - ports and adapters pattern, 82
 - monitoring element relations, 109
 - most responsible moment, 75
 - moth ball document, 186
 - Mugan's Curiosity Cycle, 102
 - multi-tier pattern, 92
 - Mural, 196, 300
 - Mythical Man Month*, 58
- ## N
-
- names
 - in business goal statements, 44
 - code reviews, 291
 - concepts, 105, 237, 272
 - elements, 105
 - importance of good, 105
 - merging meta-models, 104
 - models, 100, 104–105
 - modular decomposition diagrams, 272
 - patterns, 139
 - relations, 237
 - stages of naming, 105
 - stakeholder maps, 222
 - NASA Mars Rover, 53
 - Netflix, 297
 - Newsies*, 282
 - No Silver Bullet*, 187
 - notations
 - architecture descriptions, 151
 - context diagrams, 265
 - diagrams, 136, 138–139, 151, 265, 278, 280
 - round-robin design activity, 254
 - sequence diagrams, 278, 280
 - viewpoints, 153
 - Notes on the Synthesis of Form*, 28, 225
 - notional architecture
 - design plan, 37
 - inception deck, 270
 - influential functional requirements, 56
- ## O
-
- The Object Primer: Agile Model-Driven Development with UML 2.0*, 232
 - Object-Oriented Software Construction*, 109
 - observing behavior activity, 295–297
 - omniscient concepts, 237
 - open source contribution pattern, 95
 - open/closed principle, 76
 - oral history, 145
 - outcome in business goal statements, 44, 218
 - ownership
 - architecture briefings, 286
 - open source contribution pattern, 95
 - promoting architectural thinking, 178
 - question-comment-concern activity, 298
 - round-robin design activity, 253
 - Ozkaya, Ipek, xvii, 161

P

- pair design/programming, 112, 180
- parking lots, 124
- partitioning
 - layers pattern, 80–81
 - system, 4, 12, 185
- passive design, 36
- paths not taken, 155, 228, 231, 274
- A Pattern Language: Towns, Buildings, Construction*, 17
- Pattern Languages of Program Design* 4, 96
- patterns, architecture, 79–97
 - advantages, 79
 - architectural mismatch, 88
 - architecture haiku, 263
 - big ball of mud pattern, 96
 - building model into code, 108
 - center of competence pattern, 93
 - compared to system metaphors, 282
 - defined, 79
 - vs. design patterns, 80
 - diagrams, 138–139
 - discovering new, 96
 - layers pattern, 80–81
 - meta-model and, 104
 - models, 104, 108, 111
 - multi-tier pattern, 92
 - names, 139
 - observing behavior activity, 296
 - open source contribution pattern, 95
 - organizing code, 108
 - pipe-and-filter pattern, 84
 - ports and adapters pattern, 82
 - promoting quality attributes, 79
 - publish-subscribe pattern, 69–71, 86, 88
 - quality attribute scenarios, 80
 - quality attributes, 69–71, 80
 - selecting, 69–71
 - service-oriented pattern, 86, 297
 - shared-data pattern, 90
 - patterns, design, 80
- peer reviews, *see also* code reviews
 - architecture decision records, 260
 - scaffolding, 180
- performance, *see* quality attributes
- personify the architecture, 120, 184, 226
- pipe-and-filter pattern, 84
- pitching in architecture design studios, 117
- planning and design sweet spot, 29–31
- platform-as-a-service, 110
- Point-of-View (POV) mad lib, 44, 215–218
- Poker, Delegation, 184
- policies, design, 180
- Poppendieck, Mary, 75
- Poppendieck, Tom, 75
- portfolios, 11
- ports and adapters pattern, 82
- poster activity, group, 119, 249–252
- POV (Point-of-View) mad lib, 44, 215–218
- practice
 - drawing and sketching, 120
 - safe practice and team development, 179–181
- presentations, 149, 214
- problems
 - activities for understanding, 191–223
 - in architectural issues rainbow, 172
 - defining as role of software architects, 4
 - discovering new patterns and, 97
 - evolution of problem solving, 99
 - routine, 28
 - selecting architecture and, 65
 - simplifying as design strategy, 28
 - thinking about solution at same time, 28, 225
- producers in publish-subscribe pattern, 88
- programmers, *see also* team development
 - transitioning to software architect, 4, 11–12, 175, 177–187
 - using this book, xv
- Project Lionheart
 - about, xvii
 - architecturally significant requirements (ASRs), 50, 54–55, 62, 77
 - architecture decision records, 262
 - architecture descriptions, 153, 155–156
 - architecture design studio, 126
 - architecture evaluation, 162–164, 175
 - architecture haiku, 264
 - business goals, 44, 46, 218
 - cartoon, 135
 - concept map, 238
 - conclusion, 186
 - constraints, 50, 77
 - context diagrams, 266
 - decision matrix, 72–73, 294
 - design hills, 217
 - design rubric, 162–164
 - design strategy, 38
 - documentation, 186
 - models, 111
 - paths not taken, 155
 - patterns, 97, 111
 - Point-of-View (POV) mad lib, 216
 - quality attribute scenarios, 54–55, 175, 309–310
 - scenario walkthrough, 309–310
 - selecting architecture, 72–73, 77
 - setup, 14
 - stakeholders, 40–43, 46
 - stakeholders map, 41–43
 - system metaphors, 282
 - viewpoints, 153, 187
 - views and visualization, 130–135, 141, 175
- properties
 - decision matrix, 71–73
 - models, 100
 - system metaphors, 281, 283

prototypes, 41, 276, *see also* models
 publish-subscribe pattern, 69–71, 88, 296
 publishers
 publish-subscribe pattern, 88
 service-oriented architecture pattern, 86

Q

quality attribute scenarios, *see also* raw scenarios
 activities, 59
 architecture evaluation, 163, 175
 architecture flipbook, 229
 Component Responsibility Cards (CRC cards), 233
 concept map, 237
 decision matrix, 72–73, 292
 empathy maps, 196
 exercise, 55
 Goal-Question-Metric (GQM) Workshop, 199–201
 interviewing stakeholders, 202
 mini-Quality Attribute Workshop (mini-QAW), 210–214
 patterns, 80
 Project Lionheart, 54–55, 175, 309–310
 quality attribute web, 207–209
 response measures, 53, 55, 204, 214, 219
 scenario walkthrough, 168, 184, 307–310
 templates, 211
 using, 52–56
 views, 132
 quality attribute web, 198, 207–209, 211, 213
 Quality Attribute Workshops (QAWs), 214, *see also* mini-Quality Attribute Workshop (mini-QAW)
Quality Attribute Workshops (QAWs), Third Edition, 214
 quality attributes, *see also* quality attribute scenarios
 activities, 59, 191, 193
 architectural mismatch and, 88
 architecture descriptions, 144
 architecture design studios, 115
 architecture evaluation, 162, 166
 architecture haiku, 263
 architecture, selecting, 68–73
 conceptual constraints, 104
 defined, 10, 49, 51
 defining, 51–56
 empathy maps, 196
 exercises, 55
 frameworks and technology and, 65
 vs. functional requirements, 52
 mini-Quality Attribute Workshop (mini-QAW), 59, 210–214, 220
 models, 100, 104, 107, 109
 modular decomposition diagrams, 272
 organizing code to make patterns obvious, 109
 partitioning the system, 5
 patterns, 69–71, 80
 preserving ambiguity, 17
 promotion with architecture, 7, 10, 13, 68–73, 79, 107
 quality attribute web, 198, 207–209, 211, 213
 Quality Attribute Workshops (QAWs), 214
 sketch and compare activity, 311–313
 software architect's role, 4, 10, 186
 structures and, 10, 51
 taxonomy, 208, 213
 trade-offs, 6, 186
 viewpoints, 154
 views, 132
 quality requirements, *see* quality attributes
 question-comment-concern activity, 168, 175, 184, 298–300
 questions
 architecture briefings, 287
 architecture design studios, 114, 117
 architecture evaluation, 166, 170, 175
 curiosity cycle, 103
 divide and conquer activity, 240
 event storming, 244–248
 Goal-Question-Metric (GQM) Workshop, 59, 199–201, 296
 inception deck, 269
 interviewing stakeholders, 202–204
 mini-Quality Attribute Workshop (mini-QAW), 212
 question-comment-concern activity, 168, 175, 184, 298–300
 scenario walkthrough, 308
 in traditional software architecture description (SAD), 148
 quick checks in architecture evaluation, 171

R

rainbow, architectural issues, 172–175
 Rasmusson, Jonathan, 269, 271
 ratings, design rubrics, 162, 164
 rationale, *see* design rationale
 rationality, bounded, 27
 raw scenarios
 defined, 54
 mini-Quality Attribute Workshop (mini-QAW), 210–214
 quality attribute web, 207–209
 response measure straw man, 219
 reader in scenario walkthrough, 308
 reading in shared-data pattern, 90
 reading list, greatest hits, 267
 recorder in scenario walkthrough, 307
 red/black deployment, 66

- redesign, 16–17, 65, 225
 - refinement views, 131
 - relations
 - center of competence pattern, 94
 - Component Responsibility Cards (CRC cards), 233
 - concept map, 236–238
 - creating structures, 8
 - defined, 8
 - enforcing with models, 109
 - exercise, 9
 - layers pattern, 81
 - multi-tier pattern, 92
 - names, 237
 - open source contribution pattern, 95
 - pipe-and-filter pattern, 84
 - ports and adapters pattern, 82
 - publish-subscribe pattern, 88
 - selecting architecture and, 64
 - service-oriented pattern, 86
 - shared-data pattern, 90
 - relationships, stakeholders map, 41–43, 221–223
 - reliability, *see* quality attributes
 - repositories
 - enforcing element relations, 110
 - open source contribution pattern, 95
 - requirements, *see* architecturally significant requirements (ASRs); influential functional requirements
 - resources
 - for this book, xvii
 - on communication, 61
 - response, 53
 - response measure straw man, 55, 204, 214, 219
 - response measures, 53, 55, 204, 214, 219
 - response messages in sequence diagrams, 279
 - responsibilities
 - architect's role in assigning, 4, 185
 - in architecture descriptions, 149
 - assigning to elements, 73–75
 - center of competence pattern, 94
 - Component Responsibility Cards (CRC cards), 75, 232–235
 - element names, 105
 - element-responsibility catalog, 74, 130
 - element-responsibility views, 130
 - last responsible moment, 75
 - most responsible moment, 75
 - open source contribution pattern, 95
 - sanity check, 304
 - selecting architecture and, 64
 - single, 76
 - whiteboard jams, 257
 - Responsibility Area in center of competence pattern, 94
 - Responsible for in center of competence pattern, 94
 - REST in service-oriented pattern, 86
 - reviewers in scenario walk-through, 308
 - reviews, *see* code reviews; critiques; peer reviews
 - rework and design sweet spot, 29–31
 - risk
 - architectural issues rain-bow, 172
 - architecture evaluation, 170, 172
 - defined, 33
 - design plan, 37
 - design strategy, 28, 32–37
 - design sweet spot, 29
 - divide and conquer activity, 240
 - event storming, 244–248
 - inception deck, 270
 - passive design, 36
 - Project Lionheart, 38
 - question-comment-concern activity, 298
 - risk statements, 33
 - risk storming, 168, 184, 301–303
 - in traditional software architecture description (SAD), 148
 - risk statements, 33
 - risk storming, 168, 184, 301–303
 - robot example of quality attribute scenarios, 53
 - rolling upgrades, selecting architecture and, 66
 - Rombach, H. Dieter, 199
 - rot, architectural, *see* architectural drift
 - round-robin design activity, 119, 125, 252–254
 - Rozanski, Nick, 154
 - rubrics
 - architecture evaluation, 160, 162–164, 167
 - decision matrix, 293
 - rules
 - HART principles, 15–18
 - meta-model, 101–107
 - runtime
 - component and connector structures, 8–9
 - contracts, 109
 - multi-tier pattern, 92
 - observing behavior, 295–297
 - sequence diagrams, 278
 - service-oriented patterns and, 86
- ## S
- SAD (software architecture description), *see* architecture descriptions; traditional software architecture description (SAD)
 - sanity check, 252, 298, 304–306, 312
 - satisficing design, 27–29, 285
 - SATURN software architecture conference, xiii
 - scaffolding, 180
 - scalability, *see* quality attributes
 - scenario walkthroughs, 168, 184, 307–310
 - scenarios, *see* quality attribute scenarios; raw scenarios

- The Sciences of the Artificial*, 27
- scope and inception deck, 270
- sequence diagrams, 278, 280
- service registry, 86
- service-oriented architecture pattern, 69–71, 86, 297
- services in service-oriented pattern, 86
- shared-data pattern, 90
- sharing
 - architecture design studios, 115, 117
 - create-share-critique cycle, 117, 256
 - round-robin design activity, 252–254
- show-and-tell meeting, 241
- sign-off evaluation, 160
- silver toolbox, xvi, 187
- Simian Army, 297
- Simon, Herbert, 27
- simplicity
 - simplifying problems in design strategy, 28
 - visualizations, 139
- sink elements in pipe-and-filter pattern, 84
- size and design strategy, 30
- sketch and compare activity, 168, 311–313
- sketching, *see* drawings and sketches
- skills
 - facilitating team development, 179, 183
 - scenario walkthrough, 309
 - soft, 183
- sliders, trade-off, 194
- slideware architecture description, 149
- SOA, *see* service-oriented architecture pattern
- SOAP, 86
- soft skills, 183
- software architects, *see* architects
- software architecture, *see* architecture
- software architecture description (SAD), *see* architecture descriptions; traditional software architecture description (SAD)
- software architecture design, *see* design
- Software Architecture for Developers*, 109, 301
- Software Architecture in Practice*, 8, 51, 170
- Software Engineering Institute, 147, 154, 211
- SOLID principles, 76
- solutions
 - activities for exploring, 225–257
 - discovering new patterns and, 97
 - evolution of problem solving, 99
 - inception deck, 270
 - thinking about problems at same time, 28, 225
 - treating as experiments, 28
- source elements in pipe-and-filter pattern, 84
- source in quality attribute scenarios, 52
- Splunk, 297
- stakeholder groups, 39
- stakeholders, *see also* design mindsets; design strategy
 - active listening, 61
 - activities, 59, 191–194
 - architecture briefings, 286–288
 - architecture descriptions, 144, 147, 149–155
 - architecture design studios, 115, 121
 - architecture evaluation workshops, 167
 - architecture haiku, 263
 - business goals, 43–46, 59
 - concept map, 236
 - constraints and, 67
 - context diagrams, 265
 - curiosity cycle, 103
 - customer-centric design, 40
 - decision matrix, 72, 292
 - defined, 39
 - delegating authority, 184
 - design hills, 217
 - divide and conquer activity, 240
 - empathizing with, 16, 39–46, 191, 195–198
 - empathy maps, 195–198
 - exercises, 43, 46
 - Goal-Question-Metric (GQM) Workshop, 199–201
 - greatest hits reading list, 267
 - group poster activity, 249–251
 - groups, 39
 - identifying ASRs, 59
 - inception deck, 269–271
 - interviewing, 59, 202–204, 220
 - map, 40–43, 221–223
 - mapping views, 134
 - mini-Quality Attribute Workshop (mini-QAW), 210–214, 220
 - models, 103
 - naming, 222
 - Point-of-View (POV) mad lib, 215–218
 - Project Lionheart, 38, 40–43, 46
 - prototypes, 276
 - quality attribute web, 207–209
 - response measure straw man, 55, 219
 - round-robin design activity, 253
 - scenario walkthrough, 308
 - tangibles and, 259
 - technical debt, 6
 - trade-offs, 6–7, 192–194
 - traditional software architecture description (SAD), 147
 - views, 129, 148, 152–155
 - whiteboard jams, 255–257
- static analysis tools, 109–110
- stimulus in quality attribute scenarios, 52
- stopping conditions, 37
- stories and story telling
 - architecture descriptions, 145, 150
 - architecture flipbook, 229
 - diagrams, 140
 - organizing code as story, 150

- personify the architecture, 226
 - scenario walkthrough, 307–310
 - system metaphors, 281–283
 - user odyssey, 215
 - user story mapping, 248
 - strategy, *see* design strategy
 - straw man, response measure, 55, 204, 214, 219
 - structures, *see also* allocation structures; component and connector structures; module structures; patterns, architecture
 - creating, 8
 - defined, 8
 - enforcing element relations, 109
 - exercise, 9
 - group poster activity, 250
 - mixing, 9
 - models, 100
 - organizing code, 108
 - quality attributes and, 10, 51
 - selecting architecture, 63–77
 - sequence diagrams, 278
 - system metaphors, 281
 - system shape, 9
 - subject in business goal statements, 44, 218
 - subscribers
 - publish-subscribe pattern, 88
 - service-oriented architecture pattern, 86
 - Succeeding with Agile: Software Development Using Scrum*, 171
 - sweet spot, design, 29–31
 - synchronous request messages in sequence diagrams, 279
 - system
 - context, 5, 139
 - modular decomposition diagrams, 272
 - partitioning, 4, 12, 185
 - risk storming, 301–303
 - sequence diagrams, 278
 - shape and structures, 9
 - system metaphors, 146, 184, 281–283
 - System Context diagram, 139
 - system metaphors, 146, 184, 281–283
- ## T
- tables, 134
 - tangibility rule
 - activities to make design tangible, 259–283
 - architecture descriptions, 144
 - defined, 16
 - importance of, 17
 - make design mindset, 19
 - stakeholders and, 259
 - tangible artifacts for architecture evaluation, 160
 - targeted evaluation, 171
 - taxonomy, quality attribute, 208, 213
 - team development, 177–187
 - architecture design studios, 115, 118, 120–122
 - authority, 179, 181–185
 - code reviews, 289–291
 - delegating, 181–185
 - divide and conquer activity, 239–243
 - exercise, 184
 - portfolios, 11
 - promoting architectural thinking, 177
 - remote teams, 124
 - safe practice, 179–181
 - sanity check, 304–306
 - scenario walkthrough, 309
 - software architect's role, xvi, 3, 7, 185–186
 - transition to software architect, 4, 11–12, 175, 177–187
 - team organization
 - architecture descriptions, 144
 - center of competence pattern, 94
 - Conway's Law, 58
 - mapping views, 134
 - modular decomposition diagrams, 272
 - open source contribution pattern, 95
 - stakeholder map, 221–223
 - traditional software architecture description (SAD), 148
 - technical constraints, 50, 66
 - technical debt, 6, 186
 - technology, selecting architecture and, 65
 - tell-show-tell approach, 123
 - templates
 - architecture decision records, 260–261
 - architecture descriptions, 152
 - architecture haiku, 264
 - Point-of-View (POV) mad lib, 215–218
 - quality attribute scenarios, 211
 - scaffolding, 180
 - traditional software architecture description (SAD), 147
 - test pyramid, 171
 - testing, *see also* architecture evaluation
 - observing behavior activity, 295
 - response measures, 55
 - tangibles, 259
 - test pyramid, 171
 - testability in code reviews, 291
 - think, do, check loop, 21–24, 35, 77
 - thrashing, 31
 - time
 - architecture design studios, 113, 115, 117, 122, 124–125
 - brainstorming, 213
 - design strategy, 29–32, 36
 - design sweet spot, 29–31
 - divide and conquer activity, 239
 - models, 101
 - prototypes, 276
 - risk and design strategy, 36
 - scenario walkthrough, 309
 - sequence diagrams, 278

- time line for design plan, 37
 - time line in inception deck, 270
 - trade-offs
 - activities, 73, 192–194
 - architecture haiku, 263
 - Architecture Trade-off Analysis Method (ATAM), 170
 - decision matrix, 71–73, 292
 - defined, 63
 - inception deck, 270
 - quality attributes, 6, 186
 - sliders, 194
 - stakeholders, 6–7, 192–194
 - traditional software architecture description (SAD), 147, 161, *see also* architecture descriptions
 - tribal architecture description method, 145
 - Turner, Richard, 32
 - Tyree, Jeff, 262
- U**
-
- UML (Unified Modeling Language), 136, 138–139, 151
 - understand design mindset
 - activities, 191–223
 - defined, 18
 - exercise, 20
 - using, 18–24
 - using risk to choose, 34
 - Unified Modeling Language (UML), 136, 138–139, 151
 - unknowns in architectural issues rainbow, 172
 - updates and upgrades
 - merging meta-models, 104
 - selecting architecture and, 66
 - upstream repository workflow
 - and enforcing element relations, 110
 - user experience, 41, 248
 - user odyssey, 215
 - user stories
 - architecture flipbook, 229
 - user story mapping, 248
 - user story mapping, 248
 - Using Risk to Balance Agile and Plan-Driven Methods*, 32
 - utility trees, 162
- V**
-
- Van Heesch, Uwe, 262
 - Velichansky, Michail, 281
 - version control
 - architecture decision records, 261
 - code reviews, 290
 - enforcing element relations, 110
 - viewpoints, 153–155, 187
 - views
 - architecture descriptions, 149, 152–155
 - architecture evaluation, 175
 - cartoons, 134
 - combining, 133
 - custom, 136
 - defined, 129
 - deployment views, 133
 - element-responsibility views, 130
 - mapping views, 133
 - Project Lionheart, 130–135, 141, 175
 - quality attribute views, 132
 - question-comment-concern activity, 298
 - refinement views, 131
 - risk storming, 301
 - in traditional software architecture description (SAD), 148
 - using different views in visualizations, 129–136
 - viewpoints, 153–155, 187
 - work assignment views, 133
 - Views and Beyond*, 147
 - visualizing design, 129–141, *see also* drawings and sketches; models; views
 - annotations, 130, 133, 135
 - Architecture Description Languages (ADLs), 141
 - cartoons, 134
 - decision matrix, 71–73, 292–294
 - descriptive prose, 140
 - drawing tips, 136–140
 - event storming, 244
 - highlighting patterns in, 138
 - quality attribute web, 207–209
 - question-comment-concern activity, 298–300
 - sketch and compare activity, 168, 311–313
 - tables, 134
 - using different views, 129–136
- vocabulary**
- architecture descriptions, 144, 151
 - architecture haiku, 264
 - building model into code, 108
 - concept map, 236
 - meta-model, 101
 - models and, 100–101
 - patterns and, 139
 - software architecture design benefits, 13
 - viewpoints, 153
- volatility and design strategy, 31
- W**
-
- walkthroughs, scenario, 168, 184, 307–310
 - wasteful architecture description method, 145, 149
 - web, quality attribute, 198, 207–209, 211, 213
 - whiteboard jams, 120, 255–257
 - Who Needs an Architect?*, 13
 - Woods, Eóin, 154
 - work assignment views, 133
 - workshops
 - architecture evaluation workshops, 166–170
 - Goal-Question-Metric (GQM) Workshop, 59, 199–201, 296
 - mini-Quality Attribute Workshop (mini-QAW), 59, 210–214, 220
 - writing in shared-data pattern, 90
- Y**
-
- Yoder, Joseph, 96
- Z**
-
- Zweifel, Thomas D., 61

Level Up

From daily programming to architecture and design, level up your skills starting today.

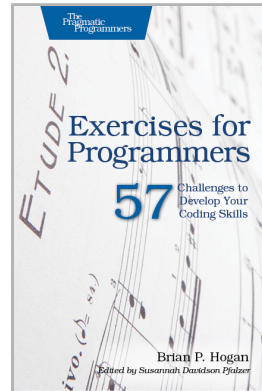
Exercises for Programmers

When you write software, you need to be at the top of your game. Great programmers practice to keep their skills sharp. Get sharp and stay sharp with more than fifty practice exercises rooted in real-world scenarios. If you're a new programmer, these challenges will help you learn what you need to break into the field, and if you're a seasoned pro, you can use these exercises to learn that hot new language for your next gig.

Brian P. Hogan

(118 pages) ISBN: 9781680501223. \$24

<https://pragprog.com/book/bhwb>



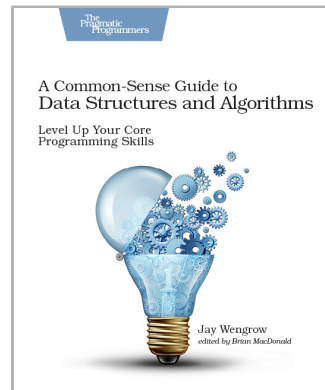
A Common-Sense Guide to Data Structures and Algorithms

If you last saw algorithms in a university course or at a job interview, you're missing out on what they can do for your code. Learn different sorting and searching techniques, and when to use each. Find out how to use recursion effectively. Discover structures for specialized applications, such as trees and graphs. Use Big O notation to decide which algorithms are best for your production environment. Beginners will learn how to use these techniques from the start, and experienced developers will rediscover approaches they may have forgotten.

Jay Wengrow

(218 pages) ISBN: 9781680502442. \$45.95

<https://pragprog.com/book/jwdsal>



Python for Everyone

For data science and basic science, for you and anyone else on your team.

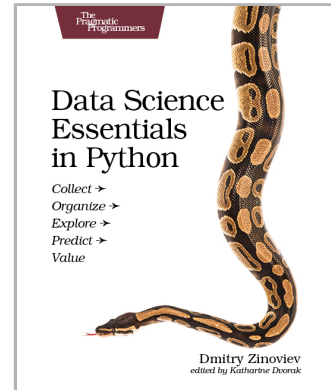
Data Science Essentials in Python

Go from messy, unstructured artifacts stored in SQL and NoSQL databases to a neat, well-organized dataset with this quick reference for the busy data scientist. Understand text mining, machine learning, and network analysis; process numeric data with the NumPy and Pandas modules; describe and analyze data using statistical and network-theoretical methods; and see actual examples of data analysis at work. This one-stop solution covers the essential data science you need in Python.

Dmitry Zinoviev

(224 pages) ISBN: 9781680501841. \$29

<https://pragprog.com/book/dzpyds>



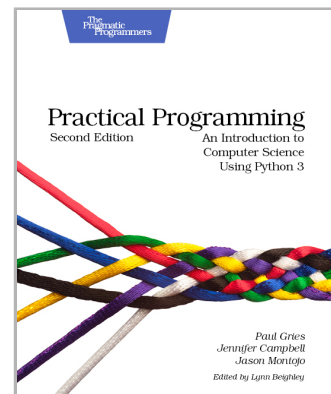
Practical Programming (2nd edition)

This book is for anyone who wants to understand computer programming. You'll learn to program in a language that's used in millions of smartphones, tablets, and PCs. You'll code along with the book, writing programs to solve real-world problems as you learn the fundamentals of programming using Python 3. You'll learn about design, algorithms, testing, and debugging, and come away with all the tools you need to produce quality code. In this second edition, we've updated almost all the material, incorporating the lessons we've learned over the past five years of teaching Python to people new to programming.

Paul Gries, Jennifer Campbell, Jason Montojo

(400 pages) ISBN: 9781937785451. \$38

<https://pragprog.com/book/gwpy2>



Explore Testing

Explore the uncharted waters of exploratory testing and delve deeper into web testing.

Explore It!

Uncover surprises, risks, and potentially serious bugs with exploratory testing. Rather than designing all tests in advance, explorers design and execute small, rapid experiments, using what they learned from the last little experiment to inform the next. Learn essential skills of a master explorer, including how to analyze software to discover key points of vulnerability, how to design experiments on the fly, how to hone your observation skills, and how to focus your efforts.

Elisabeth Hendrickson

(186 pages) ISBN: 9781937785024. \$29

<https://pragprog.com/book/ehxta>



The Way of the Web Tester

This book is for everyone who needs to test the web. As a tester, you'll automate your tests. As a developer, you'll build more robust solutions. And as a team, you'll gain a vocabulary and a means to coordinate how to write and organize automated tests for the web. Follow the testing pyramid and level up your skills in user interface testing, integration testing, and unit testing. Your new skills will free you up to do other, more important things while letting the computer do the one thing it's really good at: quickly running thousands of repetitive tasks.

Jonathan Rasmusson

(256 pages) ISBN: 9781680501834. \$29

<https://pragprog.com/book/jrtest>



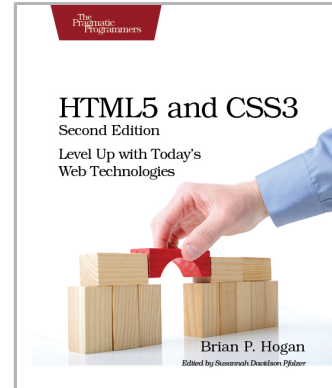
The Modern Web

Get up to speed on the latest HTML, CSS, and JavaScript techniques, and secure your Node applications.

HTML5 and CSS3 (2nd edition)

HTML5 and CSS3 are more than just buzzwords – they’re the foundation for today’s web applications. This book gets you up to speed on the HTML5 elements and CSS3 features you can use right now in your current projects, with backwards compatible solutions that ensure that you don’t leave users of older browsers behind. This new edition covers even more new features, including CSS animations, IndexedDB, and client-side validations.

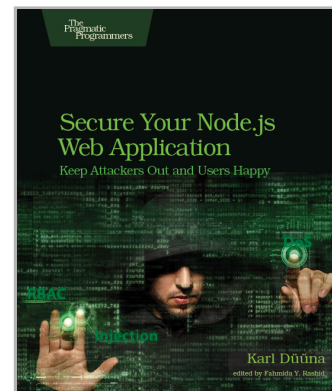
Brian P. Hogan
(314 pages) ISBN: 9781937785598. \$38
<https://pragprog.com/book/bhh52e>



Secure Your Node.js Web Application

Cyber-criminals have your web applications in their crosshairs. They search for and exploit common security mistakes in your web application to steal user data. Learn how you can secure your Node.js applications, database and web server to avoid these security holes. Discover the primary attack vectors against web applications, and implement security best practices and effective countermeasures. Coding securely will make you a stronger web developer and analyst, and you’ll protect your users.

Karl Döuna
(230 pages) ISBN: 9781680500851. \$36
<https://pragprog.com/book/kdnodesec>



The Pragmatic Bookshelf

The Pragmatic Bookshelf features books written by developers for developers. The titles continue the well-known Pragmatic Programmer style and continue to garner awards and rave reviews. As development gets more and more difficult, the Pragmatic Programmers will be there with more titles and products to help you stay on top of your game.

Visit Us Online

This Book's Home Page

<https://pragprog.com/book/mkdsa>

Source code from this book, errata, and other resources. Come give us feedback, too!

Register for Updates

<https://pragprog.com/updates>

Be notified when updates and new books become available.

Join the Community

<https://pragprog.com/community>

Read our weblogs, join our online discussions, participate in our mailing list, interact with our wiki, and benefit from the experience of other Pragmatic Programmers.

New and Noteworthy

<https://pragprog.com/news>

Check out the latest pragmatic developments, new titles and other offerings.

Buy the Book

If you liked this eBook, perhaps you'd like to have a paper copy of the book. It's available for purchase at our store: <https://pragprog.com/book/mkdsa>

Contact Us

Online Orders: <https://pragprog.com/catalog>

Customer Service: support@pragprog.com

International Rights: translations@pragprog.com

Academic Use: academic@pragprog.com

Write for Us: <http://write-for-us.pragprog.com>

Or Call: +1 800-699-7764